Large Language Models as Program Synthesizers

The emergence of large language models trained on massive corpora of source code has fundamentally altered the landscape of program synthesis. Rather than relying on hand-crafted grammars or domain-specific languages, modern code-generating models learn statistical regularities across billions of lines of human-written code, enabling them to synthesize functionally correct programs from natural language specifications. This capability has profound implications for reinforcement learning research, where the ability to automatically generate reward functions, loss functions, and policy components can accelerate algorithmic discovery beyond what manual engineering or random search can achieve.

Foundations of LLM-Based Code Generation

The watershed moment for neural program synthesis arrived with Codex, a 12-billion parameter GPT-3 model fine-tuned on publicly available code from GitHub [Chen2021]. Evaluated on HumanEval, a benchmark of 164 hand-crafted programming problems measuring functional correctness from docstrings, Codex achieved a 28.8% pass@1 rate, compared to 0% for the unmodified GPT-3. With repeated sampling of 100 candidates per problem, performance rose to 70.2%, demonstrating that even imperfect generators become powerful when paired with selection mechanisms [Chen2021]. Codex served as the backbone of GitHub Copilot, bringing LLM-based code generation into mainstream software engineering and establishing that language models could serve as practical programming assistants.

DeepMind's AlphaCode extended neural code generation to the significantly harder domain of competitive programming [Li2022]. The system adopted a generate-and-filter paradigm at unprecedented scale, producing up to millions of candidate solutions per problem using a large transformer pre-trained on GitHub code and fine-tuned on competitive programming datasets. A filtering pipeline then removed approximately 99% of generated samples based on execution against example test cases, followed by clustering of remaining solutions by program behavior to select a final set of at most 10 submissions [Li2022]. In simulated evaluations on the Codeforces platform, AlphaCode achieved an average ranking in the top 54.3% of human competitors. The significance of this result lies not in superhuman performance but in demonstrating that brute-force sampling combined with intelligent filtering can compensate for the imprecision of individual model outputs, a principle that recurs throughout LLM-guided search.

The open-source community responded with StarCoder, a 15.5-billion parameter model developed by the BigCode collaboration and trained on one trillion tokens of permissively licensed code from The Stack, spanning over 80 programming languages [Li2023]. StarCoder matched or exceeded the performance of OpenAI's earlier code-cushman-001 model and achieved 40% pass@1 on HumanEval when appropriately prompted. Its release under an open license democratized access to capable code generation, enabling researchers to fine-tune and adapt code LLMs for specialized domains without dependence on proprietary APIs.

The generalist GPT-4 model further raised the ceiling. OpenAI's technical report documented a 67% zero-shot pass@1 on HumanEval [OpenAI2023], representing a substantial jump over Codex and approaching the performance levels that previously required extensive sampling. GPT-4′s coding ability was further contextualized by SWE-bench, a benchmark of 2,294 real-world GitHub issues drawn from 12 popular Python repositories [Jimenez2024]. Unlike HumanEval's self-contained function synthesis tasks, SWE-bench requires understanding repository-level context, coordinating changes across multiple files, and reasoning about complex software dependencies. Initial evaluations found that even the best models solved fewer than

4% of issues, underscoring the gap between isolated code generation and genuine software engineering [Jimenez2024].

From Code Generation to Reward Synthesis

The capacity of LLMs to generate syntactically correct and semantically meaningful code opens a particularly consequential application in reinforcement learning. The reward function, which encodes the objective an RL agent seeks to optimize, is notoriously difficult to specify by hand. Poorly designed rewards lead to reward hacking, sparse feedback, or misaligned agent behavior. Traditionally, reward engineering has been a manual, iterative process requiring deep domain expertise.

Eureka, developed by Ma et al. at NVIDIA and published at ICLR 2024, demonstrated that GPT-4 could serve as an autonomous reward designer [Ma2024]. The system operates through an evolutionary loop. GPT-4 generates candidate reward functions as executable Python code, each function is used to train an RL policy in simulation, and training statistics are fed back to the model as structured "reward reflection" summaries. GPT-4 then revises the reward functions based on this feedback, iterating until convergence. Across a suite of 29 open-source RL environments spanning 10 distinct robot morphologies, Eureka's generated rewards outperformed expert human-engineered rewards on 83% of tasks, with an average normalized improvement of 52% [Ma2024]. The most striking demonstration involved dexterous pen spinning with an anthropomorphic Shadow Hand, a task so complex that no prior automated method had achieved it. Critically, Eureka required no task-specific prompting or pre-defined reward templates. The LLM's pre-training on vast amounts of code and technical documentation endowed it with sufficient domain knowledge to reason about physical quantities, contact dynamics, and rotational mechanics.

A complementary approach, Language to Rewards by Yu et al. at DeepMind, addressed the interface between natural language task descriptions and reward parameterization for robotic skill synthesis [Yu2023]. Rather than generating reward functions from scratch, this method translates human language instructions into reward parameters that configure a pre-existing reward structure, enabling non-expert users to specify complex robot behaviors through natural language. The system demonstrated capabilities including novel locomotion gaits and manipulation skills in MuJoCo environments, highlighting how LLMs can bridge the gap between human intent and the mathematical formalism that RL algorithms require [Yu2023].

The Key Insight and Its Limitations

These results collectively illuminate a central insight. LLMs trained on code encode substantial domain knowledge about what constitutes well-structured programs, reasonable physical heuristics, and effective optimization objectives. When deployed within an iterative search loop, they function not as random mutation operators but as informed search operators, proposing modifications grounded in patterns learned from millions of human-authored programs. This is fundamentally different from the random perturbations employed by traditional evolutionary algorithms or the uninformed sampling of early program synthesis methods.

However, LLM-based code generation carries significant limitations that temper enthusiasm with necessary caution. Code hallucination, where the model generates plausible-looking but functionally incorrect code, remains pervasive. Research has identified a taxonomy of hallucination types spanning at least 12 specific categories, including references to non-existent APIs, incorrect algorithmic logic, and fabricated library functions [Liu2024]. The quality of generated code is highly sensitive to prompt formulation. Variations in prompt format, complexity, and the

inclusion or omission of contextual information can substantially alter model outputs, making reproducibility a concern [Liu2024]. Most fundamentally, LLMs provide no formal guarantees of correctness. Unlike verified synthesis methods grounded in formal logic, neural code generation is inherently probabilistic, and the risk of subtle bugs that pass superficial testing but fail on edge cases is ever-present. For safety-critical applications, this limitation demands that LLM-generated code be treated as a starting point for human review rather than a finished product.

The inability to guarantee correctness is particularly relevant in the reinforcement learning context, where a subtly flawed reward function can produce policies that appear competent during training but exhibit catastrophic failures at deployment. The field therefore faces a productive tension. LLMs dramatically accelerate the search for good RL components, but the absence of formal verification means that their outputs must always be validated empirically, an observation that motivates the evolutionary frameworks discussed in subsequent sections.

References

[Chen2021] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. de Oliveira Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, A. Ray, R. Puri, G. Krueger, M. Petrov, H. Khlaaf, G. Sastry, P. Mishkin, B. Chan, S. Gray, N. Ryder, M. Pavlov, A. Power, L. Kaiser, M. Bavarian, C. Winter, P. Tillet, F. P. Such, D. Cummings, M. Plappert, F. Chantzis, E. Barnes, A. Herbert-Voss, W. H. Guss, A. Nichol, A. Paino, N. Tezak, J. Tang, I. Babuschkin, S. Balaji, S. Jain, W. Saunders, C. Hesse, A. N. Carr, J. Leike, J. Achiam, V. Misra, E. Morikawa, A. Radford, M. Knight, M. Brundage, M. Murati, K. Mayer, P. Welinder, B. McGrew, D. Amodei, S. McCandlish, I. Sutskever, and W. Zaremba, "Evaluating Large Language Models Trained on Code," arXiv:2107.03374, 2021.

[Li2022] Y. Li, D. Choi, J. Chung, N. Kushman, J. Schrittwieser, R. Leblond, T. Eccles, J. Keeling, F. Gimeno, A. Dal Lago, T. Hubert, P. Choy, C. de Masson d'Autume, I. Babuschkin, X. Chen, P.-S. Huang, J. Welbl, S. Gowal, A. Cherepanov, J. Molloy, D. J. Mankowitz, E. Sutherland Robson, P. Kohli, N. de Freitas, K. Kavukcuoglu, and O. Vinyals, "Competition-Level Code Generation with AlphaCode," *Science*, vol. 378, no. 6624, pp. 1092–1097, 2022.

[Li2023] R. Li, L. B. Allal, Y. Zi, N. Muennighoff, D. Kocetkov, C. Mou, M. Marone, C. Akiki, J. Li, J. Chim, Q. Liu, E. Zheltonozhskii, T. Y. Zhuo, T. Wang, O. Dehaene, M. Davaadorj, J. Lamy-Poirier, J. Monteiro, O. Shliazhko, N. Gontier, N. Meade, A. Zebaze, et al., "StarCoder: May the Source Be with You!," arXiv:2305.06161, 2023.

[OpenAI2023] OpenAI, "GPT-4 Technical Report," arXiv:2303.08774, 2023.

[Jimenez2024] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. R. Narasimhan, "SWE-bench: Can Language Models Resolve Real-World GitHub Issues?," in *Proc. ICLR*, 2024.

[Ma2024] Y. J. Ma, W. Liang, G. Wang, D.-A. Huang, O. Bastani, D. Jayaraman, Y. Zhu, L. Fan, and A. Anandkumar, "Eureka: Human-Level Reward Design via Coding Large Language Models," in *Proc. ICLR*, 2024.

[Yu2023] W. Yu, N. Gileadi, C. Fu, S. Kirmani, K.-H. Lee, M. G. Arenas, H.-T. L. Chiang, T. Erez, L. Hasenclever, J. Humplik, B. Ichter, T. Xiao, P. Xu, A. Zeng, T. Zhang, N. Heess, D. Sadigh, J. Tan, Y. Tassa, and F. Xia, "Language to Rewards for Robotic Skill Synthesis," in *Proc. CoRL*, 2023.

[Liu2024] T. Liu, C. Xu, and J. Zhu, "Beyond Functional Correctness: Exploring Hallucinations in LLM-Generated Code," arXiv:2404.00971, 2024.